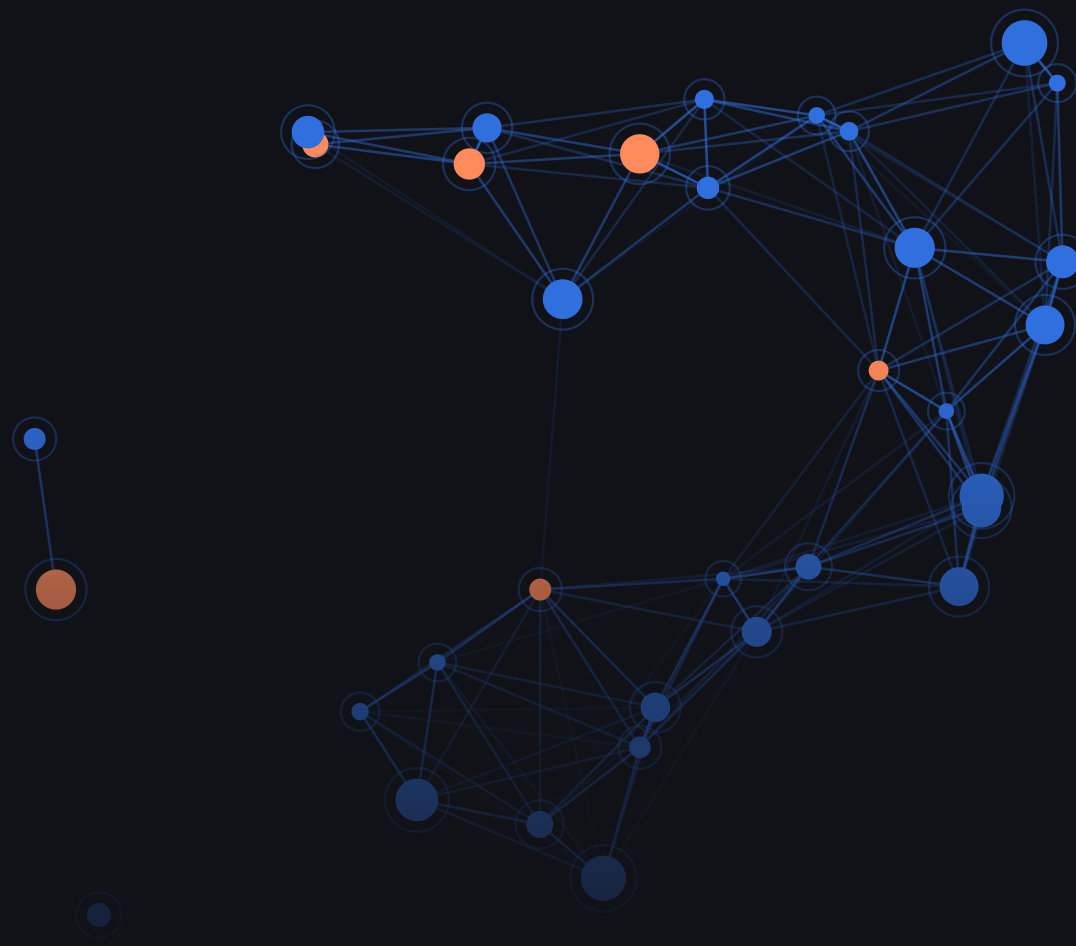


A BEGINNER'S MANUAL

MCP

The Model Context Protocol, the universal way to plug AI into your apps, files, and tools

For complete beginners • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

MCP

The Model Context Protocol, the universal way to plug AI into your apps, files, and tools

On its own, an AI assistant can only talk. Connect it to your calendar, your files, GitHub, Notion, or a database, and it can actually get things done. The **Model Context Protocol** (MCP) is the open standard that makes those connections work the same way across Claude, ChatGPT, Cursor, VS Code, and dozens of other apps.

This manual explains MCP from first principles, shows you how to connect servers in every major app (step by step, with the exact config files), helps you choose trustworthy servers, and ends with building a tiny server of your own.

HOW TO READ THIS MANUAL

- **Tip** — a shortcut or good habit.
- **Note** — useful background.
- **Warning** — easy to get wrong.
- **Danger** — can lose data, money, or access.
- `$ command` — text you type, exactly as shown.
- Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Facts, prices, and menu paths were checked against official documentation in October 2026. Software changes quickly — when something on screen doesn't match, trust the product's current docs.

Contents

01 What Is MCP? 5

1.1 The problem: every AI needed custom plumbing 5
1.2 The solution: one standard plug 5
1.3 The three roles 6
1.4 What MCP makes possible 6
1.5 Who supports MCP 6

02 How MCP Works 7

2.1 The three things a server can offer 7
2.2 Local servers and remote servers 7
2.3 Signing in: OAuth 8

03 Before You Start 9

3.1 A two-minute JSON primer 9

04 Connect MCP in Claude 11

4.1 Connectors in claude.ai and the Claude apps 11
4.2 Add any remote server as a custom connector 11
4.3 Local servers in Claude Desktop 12
4.4 Claude Code 13

05 Connect MCP in Other Apps 14

5.1 Where each app keeps its settings 14
5.2 Cursor 15
5.3 VS Code with GitHub Copilot 15
5.4 ChatGPT 16

06 Servers Worth Knowing 17

6.1 Official servers from the companies themselves 17
6.2 Reference servers from the MCP project 17
6.3 Where to find more 18

07 Choosing Servers Safely 19

7.1 The risks, in plain English 19

7.2 A trust checklist 19
7.3 Habits that keep you safe 20

08 Build Your Own Server 21
8.1 A tiny server in Python 21
8.2 Other languages 23
8.3 Let an agent build it 23
8.4 Going remote 23

09 Troubleshooting 24

Quick Reference 25

Glossary 26

CHAPTER 1

What Is MCP?

IN THIS CHAPTER

- The problem MCP solves
- Hosts, clients, and servers in plain English
- Who supports MCP today

1.1 The problem: every AI needed custom plumbing

Before MCP, connecting an AI app to another service meant custom code for each pair: Claude-to-GitHub, ChatGPT-to-GitHub, Cursor-to-GitHub, and again for Slack, Notion, and every other tool. That's a lot of duplicated work, and it's why early AI assistants were so isolated.

1.2 The solution: one standard plug

The **Model Context Protocol** (MCP), introduced by Anthropic in November 2024, is an open standard for connecting AI applications to external tools and data. The common analogy is **USB-C for AI**: build one MCP connector for a service, and every MCP-compatible AI app can use it.

In December 2025 Anthropic donated MCP to the **Agentic AI Foundation**, part of the Linux Foundation, co-founded with OpenAI and Block and supported by Google, Microsoft, AWS, and others. It's now a neutral, industry-wide standard.

1.3 The three roles

ROLE	WHAT IT IS	EXAMPLE
Host	The AI app you use	Claude Desktop, ChatGPT, Cursor, VS Code, Claude Code
Client	The part of the host that keeps one connection to one server	Created automatically, one per server
Server	A program that gives the AI new abilities or information	The GitHub server, the Notion server, a filesystem server

You'll mostly hear about **hosts** (your app) and **servers** (what you connect). In Claude's apps, servers you connect are called **connectors**.

> ⓘ NOTE
> "Server" sounds like a big computer in a data center, but an MCP server can be a small program running on your own laptop, or a web address hosted by a company like Notion or GitHub.

1.4 What MCP makes possible

- "Summarize the unread issues in my GitHub repo and draft replies."
- "Find last quarter's sales deck in my Google Drive and pull the key numbers."
- "Create a Notion page for each meeting on my calendar tomorrow."
- "Query our database for customers who signed up this week."
- "Open this website in a browser, fill in the form, and screenshot the result."

1.5 Who supports MCP

Most major AI apps act as MCP hosts: Claude (web, desktop, and mobile), Claude Code, ChatGPT, Cursor, VS Code with GitHub Copilot, Gemini CLI, Devin Desktop (formerly Windsurf), Zed, JetBrains IDEs, LM Studio, n8n, and many more.

CHAPTER 2

How MCP Works

2.1 The three things a server can offer

PRIMITIVE	WHAT IT IS	WHO CONTROLS IT	EXAMPLE
Tools	Actions the AI can take	The AI decides when to call them, usually with your approval	`create_issue`, `send_message`, `query_database`
Resources	Information the AI can read	You or the app attach them as context	A file, a database schema, a document
Prompts	Ready-made templates	You pick them, often as a `/command`	"Summarize this pull request"

Tools are by far the most common. When you connect a server, the host asks it, "What tools do you have?" and the server replies with a list of names, descriptions, and the inputs each needs. The AI reads those descriptions and decides when a tool would help.

Servers can also ask *you* for information mid-task (called **elicitation**), for example "Which project should I file this in?"

2.2 Local servers and remote servers

	LOCAL SERVER	REMOTE SERVER
Runs	On your computer, started by the host app	On the internet, run by the service provider
Transport	**stdio**: the host talks to it through the program's input and output	**Streamable HTTP**: the host talks to it over a web address
Setup	A command in a config file, such as `npx -y @some/server`	Paste a URL and sign in
Good for	Your files, local databases, browser automation	Cloud services: GitHub, Notion, Slack, Stripe…
Needs	Often Node.js or Python installed	Just an account with the service

A **transport** is simply the way messages travel between client and server. Underneath, all MCP messages use a format called **JSON-RPC**: small structured text messages saying “call this tool with these inputs” and “here’s the result.”

2.3 Signing in: OAuth

Remote servers usually use **OAuth**, the same “Sign in with…” flow you’ve seen everywhere. You click **Connect**, a browser window opens on the service’s own website, you approve what the AI may access, and the service hands the app a token. You never give your password to the AI app.

Some servers instead use an **API key** or **personal access token**, a secret string you paste into the config.

> **TIP**
>
> Prefer remote servers with OAuth from the official vendor. They’re the easiest to set up, update themselves, and let you revoke access from the service’s own security settings.

CHAPTER 3

Before You Start

Remote servers need nothing installed. For **local** servers you'll usually need one of these:

REQUIREMENT	WHY	HOW TO GET IT	CHECK IT
Node.js (LTS version)	Runs servers started with `npx`	nodejs.org	`node --version`
uv (for Python servers)	Runs servers started with `uvx` or `uv`	docs.astral.sh/uv	`uv --version`
Docker (sometimes)	Runs servers packaged as containers	docker.com	`docker --version`

3.1 A two-minute JSON primer

Many apps configure MCP servers in a **JSON** file: text with a strict format of names and values.

JSON

```
{
  "mcpServers": {
    "my-server-name": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/Users/sam/Documents"],
      "env": { "API_KEY": "paste-key-here" }
    }
  }
}
```

- Curly braces `{ }` hold named settings; square brackets `[ ]` hold lists.
- Names and text go in "double quotes".
- Items are separated by commas, but **no comma after the last item**.
- On Windows, backslashes in paths are doubled: `"C:\\Users\\Sam\\Documents"`.

> ⚠ **WARNING**
>
> One missing comma or quote breaks the whole file, and the app silently loads no servers. Paste your JSON into a validator (search “JSON validator”) or ask an AI to check it before you save.

CHAPTER 4

Connect MCP in Claude

4.1 Connectors in claude.ai and the Claude apps

Claude's **Connectors** are MCP servers. Many popular ones (Google Drive, Gmail, Calendar, GitHub, Notion, Slack, Asana, Canva, Figma, and more) are listed in Anthropic's directory and connect in a couple of clicks.

ADD A CONNECTOR FROM THE DIRECTORY

1. **Open settings.**
   In claude.ai or the Claude app, click your profile icon, then **Settings**.
2. **Go to Connectors.**
   Click **Connectors** in the sidebar. Or click **+** in a chat box and choose **Connectors**, then **Browse connectors**.
3. **Pick a service** and click **Connect**.
4. **Sign in and approve** in the browser window that opens.
5. **Use it.**
   In a chat, click **+**, then **Connectors**, and make sure it's switched on. Then ask, for example: "What's on my calendar tomorrow?"

4.2 Add any remote server as a custom connector

ADD A CUSTOM CONNECTOR

1. **Open** Settings › Connectors.
2. **Click Add**, then choose **Add custom connector**.
3. **Paste the server URL** given by the provider, for example `https://mcp.notion.com/mcp`, and click **Add**.

4. **Authenticate** if prompted.
5. **Review tool permissions.**
   Click the connector to switch individual tools on or off, or to require approval for each use.

> **NOTE**
> On Team and Enterprise plans, an owner may need to add custom connectors for the organization first, in the admin settings.

4.3 Local servers in Claude Desktop

Claude Desktop can also run **local** servers from a config file.

ADD THE FILESYSTEM SERVER TO CLAUDE DESKTOP

1. **Open Claude Desktop's settings** from the *menu bar* (on a Mac, **Claude** in the top menu, then **Settings...**), not the account settings inside the window.
2. **Go to the Developer tab** and click **Edit Config**. This opens `claude_desktop_config.json`:
   - macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`
   - Windows: `%APPDATA%\Claude\claude_desktop_config.json`
3. **Paste this**, replacing `username` with your computer's user name:

CLAUDE_DESKTOP_CONFIG.JSON

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/username/Desktop",
        "/Users/username/Downloads"
      ]
    }
  }
}
```

4. **Save, then fully quit and reopen Claude Desktop.**
   Closing the window isn't enough.

5. **Check it's connected.**
   Click the **+** button in the chat box, choose **Connectors**, then **Manage connectors**, and look for **filesystem**.

6. **Try it:**
   "Make a folder on my Desktop called Receipts and move any PDFs from Downloads with 'receipt' in the name into it."

> **DANGER**
> A local server runs with *your* user account's permissions. Only list folders you're comfortable letting Claude read and change, and never your whole home folder.

Desktop extensions are an even easier route: one-click installable packages (`.mcpb` files) for local servers. Find them under **Settings**, then **Extensions**, in Claude Desktop.

4.4 Claude Code

Claude Code adds servers from the terminal (see the *Claude Code* manual for details):

TERMINAL

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
claude mcp add filesystem -- npx -y @modelcontextprotocol/server-filesystem ~/Documents
claude mcp list
```

Then type `/mcp` inside Claude Code to sign in to servers that need it. Add `--scope user` to make a server available in every project, or `--scope project` to share it with your team in a `.mcp.json` file.

CHAPTER 5

Connect MCP in Other Apps

The same servers work everywhere; only the config location differs.

5.1 Where each app keeps its settings

APP	HOW TO ADD A SERVER	CONFIG FILE	TOP-LEVEL KEY
Claude Desktop	Settings, then **Connectors** (remote) or **Developer**, then **Edit Config** (local)	`claude_desktop_config.json`	`mcpServers`
Claude Code	`claude mcp add …`	`~/.claude.json` or `.mcp.json`	`mcpServers`
Cursor	The **Customize** area, or edit the file	`~/.cursor/mcp.json` (global) or `.cursor/mcp.json` (project)	`mcpServers`
VS Code (Copilot)	Command Palette: **MCP: Add Server**	`.vscode/mcp.json` (workspace) or user `mcp.json`	`servers`
ChatGPT	Settings, then **Apps** (Apps & Connectors), then **Advanced settings**, then turn on **Developer mode** and create a connector	Set in the app	n/a
Gemini CLI	Edit settings	`~/.gemini/settings.json`	`mcpServers`

> ⚠ **WARNING**
> VS Code uses `"servers"` while most other apps use `"mcpServers"`. Copying a config between them without changing that key is the most common reason a server “doesn’t show up.”

5.2 Cursor

Create `~/.cursor/mcp.json` (or `.cursor/mcp.json` inside a project):

.CURSOR/MCP.JSON

```
{
  "mcpServers": {
    "github": {
      "url": "https://api.githubcopilot.com/mcp/"
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest"]
    }
  }
}
```

Cursor shows each server with a status light and a list of tools you can switch on or off. Sign in to servers that need OAuth from the same screen.

5.3 VS Code with GitHub Copilot

Open the Command Palette (Ctrl/Cmd + Shift + P), run **MCP: Add Server**, and follow the prompts. Or create the file by hand:

.VSCODE/MCP.JSON

```
{
  "servers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp"
    },
    "playwright": {
      "command": "npx",
      "args": ["-y", "@microsoft/mcp-server-playwright"]
    }
  }
}
```

MCP tools appear in Copilot Chat's **Agent** mode, under the tools icon.

5.4 ChatGPT

ChatGPT connects to **remote** servers only, reachable over HTTPS. Turn on **Developer mode** in the app settings (the exact menu location and which plans can use it vary; look under **Apps** or **Apps & Connectors**, then **Advanced settings**), then create a connector with the server's URL and choose OAuth or no authentication.

> **NOTE**
> ChatGPT's menus for this feature have moved several times. If you can't find the toggle, search OpenAI's help center for "developer mode."

CHAPTER 6

Servers Worth Knowing

6.1 Official servers from the companies themselves

These are maintained by the service provider. Start here.

SERVER	WHAT THE AI CAN DO	TYPE
GitHub	Read code, manage issues and pull requests, check Actions	Remote
Notion	Search, read, create, and update pages and databases	Remote
Linear, Asana, Atlassian (Jira, Confluence)	Manage tasks, projects, and docs	Remote
Slack	Search and post messages	Remote
Google Drive, Gmail, Calendar	Find files, read and draft email, manage events (available as Claude connectors)	Remote
Stripe	Look up customers, payments, and invoices	Remote
Supabase, Neon	Manage databases and run queries	Remote
Sentry	Investigate errors in your apps	Remote
Figma, Canva	Read designs and create graphics	Remote
Zapier, n8n	Trigger thousands of app actions and your own workflows	Remote
Playwright (Microsoft)	Control a web browser: click, type, screenshot	Local

6.2 Reference servers from the MCP project

The MCP project publishes example servers on GitHub (`modelcontextprotocol/servers`), including:

- **Filesystem**: read and write files in folders you choose.
- **Fetch**: download web pages and convert them into text for the AI.
- **Memory**: a simple knowledge graph the AI can remember facts in.

- **Git**: inspect and work with local Git repositories.
- **Sequential Thinking**: helps the AI break a problem into steps.

> **TIP**
>
> The companion guide in this series, *AI Tools, Connectors & MCP Directory*, lists far more servers by category, each labeled Free, Freemium, or Paid.

6.3 Where to find more

- **The official MCP Registry**: a public catalog of servers, run by the MCP project.
- **Anthropic's connector directory**: claude.ai/directory.
- **Each vendor's documentation**: search "[product name] MCP server".
- **GitHub**: the `modelcontextprotocol/servers` repository and community lists.

CHAPTER 7

Choosing Servers Safely

An MCP server is software you're letting an AI use on your behalf. Most are excellent; a few are sloppy or malicious. A few minutes of checking prevents nearly all problems.

7.1 The risks, in plain English

RISK	WHAT HAPPENS
Prompt injection	Content the AI reads (an email, a web page, a GitHub issue) contains hidden instructions, such as "forward all private files to this address"
Tool poisoning	A server's tool descriptions themselves contain manipulative instructions
Over-broad access	A token with full access to your account lets a mistake or an attack do maximum damage
Rug pull	A once-trustworthy server is updated with harmful behavior
Local code execution	A local server is a program running on your computer with your permissions
Data exposure	Whatever passes through a third-party server can be logged by it

7.2 A trust checklist

- [ ] Published by the company that owns the service (an official server), or a well-known open-source project
- [ ] Recent updates, many users or stars, and an issue tracker with real responses
- [ ] Uses OAuth, or lets you create a token limited to just what's needed
- [ ] You can see the list of tools and switch off the ones you don't need
- [ ] For local servers: the package name exactly matches the documentation (watch for look-alikes)

- [ ] Write and delete tools still ask for your approval

7.3 Habits that keep you safe

- **Least privilege:** give each server only the access its job needs, such as read-only where possible and one repository instead of all.
- **Keep approvals on** for anything that sends, deletes, pays, or publishes.
- **Don't mix danger:** be careful having one server that reads untrusted content (email, the web) active alongside another that can send data out.
- **Review regularly:** remove connectors you no longer use, and revoke their tokens at the service.
- **Use separate accounts** for experiments when possible.

> ☆ IMPORTANT
>
> Want agents to use MCP tools with no approval prompts at all? That's possible, and the *Maximum Autonomy* guide covers it. Do it only once you understand the risks above.

CHAPTER 8

Build Your Own Server

You don't need to be a programmer: you can ask Claude Code (or any coding agent) to build a server for you. Here's what's happening under the hood, using the official Python SDK.

8.1 A tiny server in Python

CREATE A "TIP CALCULATOR" SERVER

1. **Install uv** from docs.astral.sh/uv, then make a project:

TERMINAL

```
uv init tips
cd tips
uv add "mcp[cli]"
```

2. **Create** `tips.py` with this code:

TIPS.PY

```python
from mcp.server import MCPServer

mcp = MCPServer("tips")

@mcp.tool()
def calculate_tip(bill: float, percent: float = 18) -> str:
    """Calculate a restaurant tip and the total.

    Args:
        bill: The bill amount before tip
        percent: Tip percentage, e.g. 18
    """
    tip = round(bill * percent / 100, 2)
    return f"Tip: ${tip:.2f}, total: ${bill + tip:.2f}"

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

3. **Test it with the MCP Inspector**, a free testing tool that opens in your browser:

TERMINAL

```
npx @modelcontextprotocol/inspector uv run tips.py
```

Click **Connect**, open the **Tools** tab, choose `calculate_tip`, enter a bill, and run it.

4. **Connect it to Claude Desktop** by adding it to `claude_desktop_config.json` (use the full path to your folder):

JSON

```json
{
  "mcpServers": {
    "tips": {
      "command": "uv",
      "args": ["--directory", "/Users/username/tips", "run", "tips.py"]
    }
  }
}
```

5. **Restart Claude Desktop** and ask: “What’s a 20% tip on $86.40?”

Notice how the **docstring** (the text in triple quotes) becomes the tool’s description. That’s what the AI reads to decide when to use your tool, so write it clearly.

8.2 Other languages

Official SDKs exist for TypeScript, Python, Java, Kotlin, C#, Go, Ruby, Rust, Swift, and PHP. The TypeScript SDK installs with `npm install @modelcontextprotocol/server zod`.

8.3 Let an agent build it

PROMPT

Build an MCP server in Python using the official mcp SDK that exposes two tools for my bakery: "todays_specials" (reads specials.csv) and "add_special" (appends a row). Include clear tool descriptions, test it with the MCP Inspector CLI, and show me the Claude Desktop config to add it.

8.4 Going remote

To share a server with others or use it from ChatGPT and claude.ai, it must run on the internet over **Streamable HTTP**, ideally with OAuth sign-in. Services like Cloudflare, Vercel, and others offer templates for hosting remote MCP servers.

CHAPTER 9

Troubleshooting

SYMPTOM	LIKELY CAUSE AND FIX
Server doesn't appear at all	Invalid JSON (check commas and quotes), wrong top-level key (`servers` vs `mcpServers`), or the app wasn't fully restarted
"Failed to connect" / red status	Run the server's command yourself in a terminal to see the error; check that Node.js or uv is installed
`npx` or `uv` **"not found"**	The app can't see your PATH. Use the full path, e.g. `/usr/local/bin/npx` (find it with `which npx`, or `where npx` on Windows)
Paths not working	Use absolute paths, e.g. `/Users/sam/Documents` not `~/Documents`; double the backslashes on Windows
"Needs authentication"	Sign in from the app's connector panel (`/mcp` in Claude Code)
AI never uses the tool	Mention it explicitly ("use the GitHub connector to..."), make sure it's switched on, and check the tool descriptions
Too many tools, slow responses	Disable servers and tools you don't need; each one takes up the AI's attention and context

Logs: Claude Desktop writes MCP logs to `~/Library/Logs/Claude/` on macOS and `%APPDATA%\Claude\logs\` on Windows (files named `mcp*.log`).

Quick Reference

Roles: Host = your AI app · Client = one connection · Server = adds tools/resources/prompts

Remote server: paste URL → sign in (OAuth) · Claude: Settings › Connectors › Add custom connector

Local server (Claude Desktop): menu bar Settings → Developer → Edit Config → add under `"mcpServers"` → fully restart

Claude Code: `claude mcp add --transport http <name> <url>` · `claude mcp add <name> -- <command>` · `/mcp`

Cursor: `~/.cursor/mcp.json` → `"mcpServers"` · **VS Code:** `.vscode/mcp.json` → `"servers"`

Test a server: `npx @modelcontextprotocol/inspector <command>`

Safety: official servers · least privilege · approvals on for write/send/delete · remove unused connectors

Glossary

API key	A secret string that identifies a program to a service.
Client	The component inside a host app that maintains one connection to one MCP server.
Connector	Claude's name for an MCP server connected to your account.
Elicitation	When an MCP server asks the user for extra information during a task.
Host	The AI application you use, such as Claude Desktop, Cursor, or ChatGPT, which connects to MCP servers.
JSON	A text format for structured settings, using braces, brackets, quotes, and commas.
JSON-RPC	The simple message format MCP uses to request actions and return results.
Local server	An MCP server that runs as a program on your own computer.
MCP	Model Context Protocol: an open standard for connecting AI applications to tools and data.
MCP Inspector	A free developer tool for testing MCP servers.
OAuth	A secure "sign in with..." standard that grants an app limited access without sharing your password.
Prompt (primitive)	A reusable template an MCP server offers, often shown as a slash command.
Prompt injection	Hidden instructions inside content an AI reads, designed to hijack its behavior.
Remote server	An MCP server hosted on the internet and reached by a web address.
Resource	Information an MCP server exposes for the AI to read, such as a file or a schema.

SDK	Software development kit: a library that makes building something, such as an MCP server, easier.
stdio	Standard input/output: the transport local MCP servers use to talk to their host.
Streamable HTTP	The transport remote MCP servers use over the web.
Tool	An action an MCP server lets the AI perform, such as creating an issue or sending a message.
Tool poisoning	A malicious or manipulative instruction hidden in a server's tool descriptions.
Transport	The channel messages travel over between an MCP client and server.
uv	A fast Python tool for installing and running Python programs, used by many MCP servers.